# LEVELING PLATFORM — PID CONTROL SYSTEM

*Engineering Technical Brief*

Riya Joseph | B.Tech Mechatronics Engineering | Jyothi Engineering College, Thrissur
github.com/AYRA-J10/leveling_platform_pid | riyajosef10@gmail.com

October 2026

## 1. Problem Statement

A leveling platform used in defense and aerospace applications — mobile launch rails, shipborne antenna mounts, field command systems — must maintain precise angular position under two categories of continuous disturbance:

- External mechanical disturbances: wind gusts, vehicle motion, ground vibration. These apply sudden torque inputs that push the platform off its setpoint angle.
- Sensor measurement noise: IMU accelerometers and gyroscopes produce readings corrupted by Gaussian noise. Naively differentiating noisy sensor data in a PID derivative term amplifies this noise, causing actuator chatter and potential motor burnout.

The engineering challenge is to design a controller that corrects angular error rapidly while rejecting both categories of disturbance — without relying on expensive high-precision sensors or over-engineered actuators.

## 2. Mathematical Modeling

### 2.1 Plant Derivation**2.1 Plant Derivation**

The leveling platform is modeled as a rotating rigid body driven by a servo torque input τ. Applying Newton's second law for rotation:

$$J\cdot\ddot{\theta} + b\cdot\dot{\theta} = \tau$$

Where:

**J:** 0.01 kg·m² — Platform moment of inertia
**b:** 0.1 Ns/m — Viscous friction / damping coefficient
**θ:** Platform angle (rad) — the controlled variable
**τ:** Servo torque input — the control signal

Taking the Laplace transform with zero initial conditions:

$$Js^2\Theta(s) + bs\Theta(s) = T(s)$$

Rearranging to give the plant transfer function G(s):

$$G(s) = \Theta(s)/T(s) = 1 / (Js^2 + bs) = 1 / (0.01s^2 + 0.1s)$$

This is a second-order Type-1 plant (one integrator in the denominator). The integrator means the plant itself accumulates velocity — realistic for a servo-driven mechanical system with viscous damping.

### 2.2 Sensor Noise Model**2.2 Sensor Noise Model**

IMU sensor noise was modeled as additive Gaussian noise on the feedback signal:

$$\theta_measured(t) = \theta_actual(t) + \eta(t), \quad \eta \sim N(0, \sigma^2), \quad \sigma = 0.005$$

This level of noise is consistent with low-cost MEMS IMU datasheets (MPU-6050 class sensors). The noise standard deviation of 0.005 rad corresponds to approximately ±0.29° angular measurement uncertainty.

# 3. Control Architecture

## 3.1 PID Structure3.1 PID Structure

A PID controller with derivative filter was implemented. The continuous-time control law:

$$u(t) = Kp \cdot e(t) + Ki \cdot \int e(t)dt + Kd \cdot (N \cdot e(t) - N \cdot \int u_d\, dt)$$

In transfer function form, the filtered PID controller:

$$C(s) = Kp + Ki/s + Kd \cdot Ns/(s + N)$$

The derivative filter coefficient N limits the high-frequency gain of the derivative term. Without this filter, the derivative term amplifies sensor noise by a factor proportional to frequency — destroying actuator life and causing output chatter.

## 3.2 Tuning Methodology3.2 Tuning Methodology

Gains were tuned iteratively in MATLAB using the following logic:

**Kp = 1.5:** Proportional gain — sets the base correction force proportional to current error. Sized to give fast response without immediate instability.

**Ki = 0.5:** Integral gain — eliminates steady-state error by accumulating past error. Sized conservatively to avoid integral windup on the Type-1 plant.

**Kd = 0.8:** Derivative gain — provides predictive braking before overshoot occurs. Combined with N=100 filter to prevent noise amplification.

**N = 100:** Derivative filter coefficient — limits derivative high-frequency gain. Value of 100 chosen to preserve phase lead benefit while attenuating noise above ~16 Hz.

Multiple gain combinations were evaluated. Gains producing overshoot above 10% or steady-state error above 0.01 were rejected. Final gains were validated against the disturbance rejection and noise simulation scenarios.

## 3.3 Closed-Loop Architecture3.3 Closed-Loop Architecture

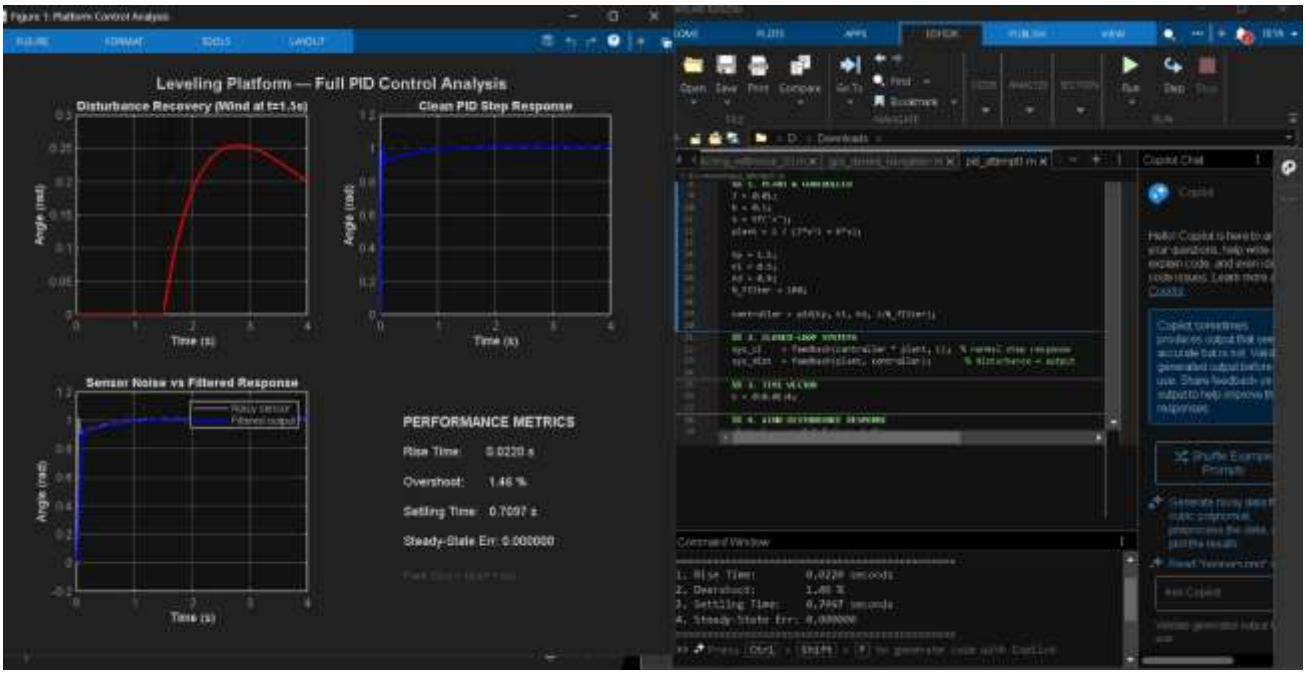
Two closed-loop configurations were simulated:

- Standard reference tracking: sys_cl = feedback(C(s)·G(s), 1) — used for step response and noise analysis.
- Disturbance rejection: sys_dist = feedback(G(s), C(s)) — models an external torque disturbance applied at the plant input, representing a wind gust.

# 4. Simulation Results

## 4.1 Performance Metrics4.1 Performance Metrics

| Metric | Value |
|---|---|
| Rise Time | **22 ms** |
| Overshoot | **1.46 %** |
| Settling Time | **709 ms** |
| Steady-State Error | **0.000000** |




These results were extracted programmatically using MATLAB's stepinfo() function and dcgain() for steady-state error verification — not read visually from plots.

### 4.2 Disturbance Rejection**4.2 Disturbance Rejection**

A step disturbance of 0.5 rad was applied at t = 1.5s, simulating a wind gust hitting the platform. The controller rejected the disturbance and returned the platform to its equilibrium position. Peak deflection under disturbance was approximately 0.25 rad, with recovery complete within 2 seconds of gust onset.

### 4.3 Noise Rejection**4.3 Noise Rejection**

Gaussian sensor noise ($\sigma = 0.005$) was added to the feedback signal. The derivative filter ($N = 100$) attenuated noise amplification in the D term. The filtered output tracked the clean step response closely, with visible noise remaining below ±0.02 rad throughout — within acceptable bounds for servo actuation.

## 5. Physical Limitations and Unmodeled Effects

The following real-world effects were deliberately excluded from this simulation. They are identified here to demonstrate awareness of the gap between simulation and hardware:

- Servo voltage saturation: Real servo actuators have a maximum torque output corresponding to their supply voltage and current limit. This simulation assumes linear, unsaturated actuation at all times. In hardware, large angular errors will saturate the servo, causing integral windup if no anti-windup clamping is implemented.
- Structural flexibility: The platform was modeled as a rigid body. Physical platforms exhibit structural resonance modes at higher frequencies. These appear as additional poles in the real transfer function and can cause instability in controllers tuned on a rigid-body model.
- Coulomb (dry) friction: The damping term $b\cdot\dot{\theta}$ models viscous friction only. Real servo joints exhibit Coulomb friction — a constant opposing torque at low angular velocities. This creates a deadband near the setpoint and can prevent true zero steady-state error in hardware even when simulation shows zero.
- IMU temperature drift: The noise model assumes stationary Gaussian noise. Real MEMS IMUs exhibit temperature-dependent bias drift. Gyroscope bias in particular grows with temperature, causing the dead-reckoning estimate to drift even after the platform is stationary.

- Discrete-time implementation delay: This simulation uses continuous-time transfer functions. On STM32 hardware, the PID loop runs at a fixed sample rate (target: 100 Hz). Discretization introduces a half-sample delay and requires z-domain re-tuning of gains for equivalent performance.

Addressing these limitations in hardware will require: anti-windup clamping on the integrator, IMU temperature compensation, discrete-time gain re-tuning, and mechanical characterization of the actual servo joint friction.